Конспект для подготовки к экзамену

Basic Machine Learning (MIPT / girafe-ai, Fall 2025)

по программе экзамена (темы 1–60 + теоретический минимум + вопросы с тестов)

Как пользоваться конспектом

- Каждый раздел — это *условный билет* (взято по пунктам программы).
- В каждом билете: определения, ключевые формулы, интуиция, мини-примеры и типичные ошибки.
- Пункты со звёздочкой **(*)** отмечены как углублённые/опциональные (я их тоже кратко разобрал).

Версия: 17 декабря 2025 г.

Содержание

1 Билет 1. Определение машинного обучения и методология 4

2 Билет 2. Применения ML и подзадачи 4

3 Билет 3. Датасеты и где их брать 5

4 Билет 4. Exploratory Data Analysis (EDA) 6

5 Билет 5. Максимальное правдоподобие (MLE) 6

6 Билет 6. Таксономия задач ML 7

7 Билет 7. Наивный байесовский классификатор 8

8 Билет 8. Библиотека `scikit-learn` и пайплайны 9

9 Билет 9. Регрессионная модель: допущения 9

10 Билет 10. Линейная регрессия: MSE, аналитическое решение, Гаусс–Марков 10

11 Билет 11. Градиентный спуск для линейной регрессии 11

12 Билет 12. Регуляризация в линейных моделях: Ridge/Lasso/Elastic Net 11

13 Билет 13. Метрики регрессии: MSE, MAE, RMSE, MAPE, SMAPE, R^2 12

14 Билет 14. Параметры и гиперпараметры 13

15 Билет 15. Валидация и оценка модели: сплиты, CV, утечки 14

16 Билет 16. Линейная классификация 15

17 Билет 17. Логистическая регрессия 16

18 Билет 18. Стратегии мультиклассовой классификации: OvR и OvO 16

19 Билет 19. Метрики классификации 17

20 Билет 20. Нормы матриц (*) 18

21 Билет 21. Support Vector Machine (SVM) 19

22 Билет 22. Principal Component Analysis (PCA) 20

23 Билет 23. k-ближайших соседей (kNN) для классификации и регрессии 20

24 Билет 24. Индексы для kNN: идеи, HNSW, векторные БД 21

25 Билет 25. The Bitter Lesson 22

26 Билет 26. Деревья решений 22

27 Билет 27. Bias–variance–noise decomposition 23

28 Билет 28. Бэггинг, бутстрэп и Random Forest 24

29 Билет 29. Обнаружение аномалий (*) 25

30 Билет 30. Градиентный бустинг 25

31 Билет 31. Библиотеки градиентного бустинга: XGBoost, LightGBM, CatBoost 26

32 Билет 32. Важности признаков и SHAP 27

33 Билет 33. Оптимизация гиперпараметров (Hyperparameter Optimization) 27

34 Билет 34. Матричный анализ: производные (минимум) 28

35 Билет 35. Концепция нейронной сети 29

36 Билет 36. Линейный слой 29

37 Билет 37. Функции активации 30

38 Билет 38. Backpropagation 31

39 Билет 39. Мультиклассовая классификация: Softmax и Cross-Entropy 31

40 Билет 40. Компьютерное железо для ML 32

41 Билет 41. PyTorch 33

42 Билет 42. Рекуррентные нейросети (RNN) 34

43 Билет 43. Ячейка LSTM и GRU 34

44 Билет 44. Многослойные RNNs 35

45 Билет 45. Операция свёртки 36

46 Билет 46. Свёрточный слой 37

47 Билет 47. Pooling layer 37

48 Билет 48. Датасет ImageNet: параметры 38

49 Билет 49. Архитектуры классификации изображений 38

50 Билет 50. Механизм внимания (Attention) 39

51 Билет 51. Архитектура Transformer 40

52 Билет 52. Geometric Machine Learning 40

53 Билет 53. Снижение размерности: IsoMap, t-SNE, UMAP 41

54 Билет 54. Кластеризация 42

55 Билет 55. Ранжирование 42

56 Билет 56. Обработка текстов (*) 43

57 Билет 57. Машинный перевод (*) 44

58 Билет 58. Экспоненциальное сглаживание (*) 44

59 Билет 59. Методы оптимизации (*) 45

60 Билет 60. Inductive bias в Deep Learning (*) 45

Теоретический минимум и вопросы с тестов 46

61 Теоретический минимум: краткие ответы 46

62 Вопросы с тестов: что отвечать в одну-две минуты 47

1 Билет 1. Определение машинного обучения и методология

Определение

Машинное обучение (ML) — это подход к построению систем, которые **улучшают качество решения задачи на данных с опытом** (данными/обратной связью), а не через ручное программирование всех правил. В типичной постановке мы выбираем семейство моделей f_θ и находим параметры θ, минимизируя функцию потерь на обучающей выборке.

На чём основано ML

- **Математика и статистика**: вероятности, оценивание параметров, статистические тесты, линейная алгебра.
- **Оптимизация**: градиентные методы, выпуклая оптимизация, регуляризация.
- **Информатика**: структуры данных, сложность, инженерия признаков, работа с пайплайнами.
- **Предметная область**: постановка задачи, сбор данных, ограничения бизнеса/продукта.

Классический код vs ML

- **Классический код**: правила фиксированы, качество меняется при изменении кода.
- **ML**: правила (параметры) получаются из данных; качество меняется при изменении данных/фич/гиперпараметров.
- В ML важно разделять: *модель* (формула/архитектура), *параметры* (обучаются) и *гиперпараметры* (задаются снаружи).

Базовые термины

- **Объект** $x \in \mathcal{X}$, **таргет/метка** $y \in \mathcal{Y}$.
- **Датасет** $\{(x_i, y_i)\}_{i=1}^n$ (для supervised) или $\{x_i\}_{i=1}^n$ (для unsupervised).
- **Модель** $f_\theta(x)$, **предсказание** $\hat{y}$.
- **Функция потерь** $\ell(\hat{y}, y)$ и **риск** $L(\theta) = \mathbb{E}[\ell(f_\theta(x), y)]$.
- **Эмпирический риск**: $\hat{L}(\theta) = \frac{1}{n}\sum_{i=1}^n \ell(f_\theta(x_i), y_i)$.

Типичная ML-методология (очень коротко)

1. Постановка задачи и метрики (что оптимизируем, как измеряем успех).
2. Сбор/разметка данных, EDA, чистка, базовые бейзлайны.
3. Обучение, валидация, подбор гиперпараметров, контроль утечек.
4. Интерпретация/ошибки, улучшения (фичи/модель/данные).
5. Деплой и мониторинг качества/дрейфа.

2 Билет 2. Применения ML и подзадачи

Идея

ML — это не одна задача, а семейство задач, где из данных извлекаются закономерности. Полезно уметь назвать подполе, тип данных и пример метрики.

Примеры подзадач (минимум 3, лучше 6–8)

- **Классификация**: спам/не спам, дефект/норма на производстве.
- **Регрессия**: прогноз цены, спроса, времени доставки.
- **Ранжирование/рекомендации**: поиск, лента, выдача товаров.
- **Кластеризация**: сегментация пользователей, группировка документов.
- **Выявление аномалий**: фрод, сбои датчиков, аномальные транзакции.
- **Компьютерное зрение**: классификация изображений, детекция объектов.
- **NLP**: анализ тональности, извлечение сущностей, перевод.
- **Временные ряды**: прогноз, детекция смен режимов, сглаживание.

Практические примеры (как рассказать на экзамене)

- **Поиск**: ранжирование документов по клику/релевантности; метрики NDCG, mAP.
- **Кредитный скоринг**: вероятность дефолта; AUC/PR-AUC, калибровка вероятностей.
- **Реклама**: прогноз CTR/конверсии; logloss, AUC; онлайн-метрики по A/B тестам.

Типичные нюансы

- В реальных продуктах часто важны **бизнес-ограничения**: стоимость ошибки, задержка, интерпретируемость, стабильность.
- Часто задача *формально* классификация, но *по смыслу* ранжирование (например, «покажи top-k»).

3 Билет 3. Датасеты и где их брать

Что считается датасетом

Набор данных — это структурированное представление наблюдений, пригодное для обучения/валидации модели: таблица (строки=объекты), коллекция изображений, тексты, граф, временные ряды и т.д.

Где искать (минимум 3 источника)

- **Публичные репозитории**: Kaggle, UCI Machine Learning Repository, OpenML.
- **Открытые государственные данные**: data.gov, data.europa.eu, Росстат и др.
- **Научные статьи/бенчмарки**: датасеты из papers + ссылки на GitHub/Zenodo.
- **Внутренние данные компании**: логи, транзакции, события, CRM (важны права доступа/PII).
- **Сбор вручную**: краулинг, API (только легально), разметка через Toloka/Label Studio.

На что смотреть при выборе

- **Семантика таргета** и качество разметки (шум, субъективность).
- **Сдвиги распределений**: train/test не должны быть из разных миров (если это не цель).
- **Размер и баланс классов**, редкость событий, стоимость ошибок.
- **Лицензия**, приватность, персональные данные, этика.

Практический совет

Всегда делайте простой бейзлайн и EDA: иногда датасет «не решает задачу» ещё до выбора модели.

4 Билет 4. Exploratory Data Analysis (EDA)

Зачем нужен EDA

- Понять данные: типы признаков, масштабы, пропуски, выбросы, баланс классов.
- Найти проблемы: утечки, дубликаты, ошибки разметки, несогласованные единицы измерения.
- Выбрать правильную метрику/сплит: например, временной сплит для временных рядов.
- Подготовить фичи: нормализация, лог-преобразования, кодирование категорий.

Типичный пайплайн EDA

1. **Схема данных**: размеры, типы колонок, доля пропусков.
2. **Описание таргета**: распределение y, дисбаланс, редкие классы.
3. **Одномерный анализ**: гистограммы/квантили, выбросы, долгие хвосты.
4. **Двумерный анализ**: связь признака с таргетом (boxplot, группировки).
5. **Корреляции/зависимости**: Pearson/Spearman, взаимная информация (осторожно с интерпретацией).
6. **Проверка утечек**: признаки «из будущего», лики по ID, агрегации, таргет-энкодинг без CV.
7. **Бейзлайн**: простая модель (логрег/дерево) + sanity-check метрик.

Примеры, которые любят на экзамене

- Дисбаланс классов $\rightarrow$ accuracy обманывает; смотреть PR-AUC, balanced accuracy, F1.
- Нелинейный таргет с «хвостом» $\rightarrow$ MSE сильно штрафует выбросы; возможно MAE/Huber.
- Признак с другим масштабом в kNN $\rightarrow$ доминирование расстояния; нужен scaling.

5 Билет 5. Максимальное правдоподобие (MLE)

Определение правдоподобия

Пусть данные $x_1, \ldots, x_n$ сгенерированы из распределения $p(x \mid \theta)$. Тогда

$$\mathcal{L}(\theta; x_{1:n}) = p(x_1, \ldots, x_n \mid \theta)$$

— **правдоподобие** параметра θ при фиксированных данных.

Следствия i.i.d.

Если x_i независимы и одинаково распределены (i.i.d.), то

$$p(x_1, \ldots, x_n \mid \theta) = \prod_{i=1}^{n} p(x_i \mid \theta), \qquad \log \mathcal{L}(\theta) = \sum_{i=1}^{n} \log p(x_i \mid \theta).$$

Это ключевой трюк: произведение превращается в сумму, удобную для оптимизации.

MLE-оценка и отрицательный лог-правдоподобия

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} \ \mathcal{L}(\theta; x_{1:n}) = \arg\max_{\theta} \ \sum_{i=1}^{n} \log p(x_i \mid \theta).$$

Часто минимизируют **NLL** (negative log-likelihood):

$$\text{NLL}(\theta) = -\sum_{i=1}^{n} \log p(x_i \mid \theta).$$

Связь с функциями потерь

Во многих моделях «потеря» — это NLL: например, MSE соответствует гауссовской модели шума, а логистическая потеря — Бернулли/категориальному распределению (см. билеты 10 и 17).

Дополнительно (*)

Если добавить априорное распределение $p(\theta)$, то получаем MAP:

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} \ \log p(\theta) + \sum_{i=1}^{n} \log p(x_i \mid \theta),$$

и регуляризация часто интерпретируется как лог-априор (см. билет 12).

6 Билет 6. Таксономия задач ML

Supervised vs Unsupervised

- **Supervised learning** (обучение с учителем): есть пары (x_i, y_i). Примеры: классификация, регрессия, ранжирование (с «правдой»), сегментация.
- **Unsupervised learning** (без учителя): есть только x_i. Примеры: кластеризация, снижение размерности, поиск аномалий (иногда без меток).

Другие категории (важно уметь назвать)

- **Semi-supervised**: мало размеченных данных + много неразмеченных.
- **Self-supervised**: метки строятся из самих данных (masked language modeling, контрастивное обучение).
- **Reinforcement learning**: агент получает награду за действия в среде.
- **Active learning**: модель выбирает, что разметить, чтобы улучшиться быстрее.
- **Online learning**: данные приходят потоком, модель обновляется постепенно.

Практические примеры (по 3+ на тип)

- Supervised: прогноз цены, распознавание спама, классификация изображений.
- Unsupervised: сегментация клиентов, поиск похожих товаров, визуализация данных в 2D.
- RL: игры, управление роботами, оптимизация стратегии показа рекламы.

Постановка практической задачи: сущности

- Объекты/признаки/таргет, целевая метрика и стоимость ошибок.
- Ограничения по данным: утечки, приватность, распределения во времени.
- Ограничения по вычислениям: время обучения/инференса, память.

7 Билет 7. Наивный байесовский классификатор

Теорема Байеса

$$p(y \mid x) = \frac{p(x \mid y)\, p(y)}{p(x)}, \qquad p(x) = \sum_{y'} p(x \mid y')p(y').$$

Для классификации достаточно сравнивать $p(y \mid x)$ по классам.

Наивное предположение

Пусть $x = (x_1, \ldots, x_d)$. **Наивный Байес** предполагает условную независимость признаков при фиксированном классе:

$$p(x \mid y) = \prod_{j=1}^{d} p(x_j \mid y).$$

Тогда апостериорная вероятность:

$$p(y \mid x) \propto p(y) \prod_{j=1}^{d} p(x_j \mid y).$$

Как обучается

- Оцениваем априоры классов: $\hat{p}(y) = \frac{\#\{i : y_i = y\}}{n}$ (частоты).
- Оцениваем параметры распределений $p(x_j \mid y)$ по данным класса y.
- На практике часто нужен **Laplace smoothing** для дискретных признаков, чтобы избежать нулевых вероятностей.

Непрерывные признаки

Типичный вариант — **Gaussian Naive Bayes**:

$$x_j \mid y \sim \mathcal{N}(\mu_{y,j}, \sigma^2_{y,j}),$$

параметры μ, σ^2 оцениваются по выборке внутри класса.

Плюсы и минусы

- Плюсы: очень быстрый, хорошо работает на текстах (bag-of-words), мало данных.
- Минусы: независимость часто нарушена $\Rightarrow$ вероятности могут быть плохо калиброваны, граница решения ограничена.

8 Билет 8. Библиотека `scikit-learn` и пайплайны

Основной интерфейс (Estimator API)

Почти всё в sklearn придерживается шаблона:

- `model = Estimator(**hyperparams)`
- `model.fit(X, y)` — обучить
- `model.predict(X)` — предсказать метку/значение
- `model.predict_proba(X)` — предсказать вероятности (если поддерживается)
- `transformer.fit(X, y=None)` и `transformer.transform(X)` — преобразования признаков

Ключевая идея: **всё похоже на всё**, поэтому удобно строить пайплайны.

Базовые классы, которые важно знать

- **Модели**: `LinearRegression`, `LogisticRegression`, `SVC`, `RandomForestClassifier`, `GradientBoosti`
- **Препроцессинг**: `StandardScaler`, `OneHotEncoder`.
- **Композиция**: `Pipeline`, `ColumnTransformer`.
- **Валидация/подбор**: `train_test_split`, `KFold/StratifiedKFold`, `cross_val_score`, `GridSearchCV/RandomizedSearchCV`.

Почему важны пайплайны

- Уменьшают риск **утечек**: препроцессинг обучается только на train.
- Удобно сравнивать модели и делать CV: один объект «модель с обработкой».
- Упрощают деплой: чёткая последовательность transform → predict.

Мини-пример (идея)

```
Pipeline([('scaler', StandardScaler()), ('clf', LogisticRegression())])
```

9 Билет 9. Регрессионная модель: допущения

Постановка

Регрессия: по объекту x предсказать вещественное значение $y \in \mathbb{R}$.

Классическая линейная регрессионная модель

$$y = f(x) + \varepsilon, \qquad \mathbb{E}[\varepsilon \mid x] = 0.$$

В линейной регрессии часто пишут $y = w^\top x + b + \varepsilon$.

Типичные допущения (для теории OLS)

- Линейность по параметрам: $y = Xw + \varepsilon$ (при фиксированных признаках X).
- **Нулевое среднее шума**: $\mathbb{E}[\varepsilon \mid X] = 0$ (несмещённость).
- **Гомоскедастичность**: $\mathrm{Var}(\varepsilon \mid X) = \sigma^2 I$ (постоянная дисперсия, нет корреляций).
- **Полный ранг** $X^\top X$ (иначе решение неединственно).

Почему важно (*)

Допущения определяют:

- корректность статистических выводов (доверительные интервалы, тесты),
- свойства оценок (несмещённость, минимальная дисперсия, устойчивость),
- выбор метрики/потери (например, гауссовский шум $\Rightarrow$ MSE).

Практические замечания

- В ML часто нас интересует **предсказательная** точность, а не строгая статистическая интерпретация.
- Нарушение гомоскедастичности/нормальности не «ломает» OLS как алгоритм, но влияет на интерпретацию и устойчивость.

10 Билет 10. Линейная регрессия: MSE, аналитическое решение, Гаусс–Марков

Модель и MSE

Пусть $X \in \mathbb{R}^{n \times d}$ — матрица признаков, $y \in \mathbb{R}^n$ — таргет, $w \in \mathbb{R}^d$. Линейная модель: $\hat{y} = Xw$ (или $Xw + b$, где b можно добавить через столбец единиц).

MSE-потеря:

$$\mathcal{L}(w) = \frac{1}{n} \|Xw - y\|_2^2 .$$

Почему MSE (интуиция и MLE*)

- MSE сильно штрафует большие ошибки, удобно дифференцируется.
- (*) Если шум ε гауссовский: $y \mid X, w \sim \mathcal{N}(Xw, \sigma^2 I)$, то MLE по w эквивалентно минимизации MSE (с точностью до константы).

Аналитическое решение (нормальные уравнения)

Градиент:

$$\nabla_w \mathcal{L}(w) = \frac{2}{n} X^\top (Xw - y).$$

Приравниваем к нулю:

$$X^\top X w = X^\top y.$$

Если $X^\top X$ обратима, то:

$$\hat{w}_{\text{OLS}} = (X^\top X)^{-1} X^\top y.$$

Теорема Гаусса–Маркова (BLUE)

Если выполнены стандартные допущения линейной модели (несмещённый шум, одинаковая дисперсия, отсутствие корреляций, фиксированный X), то OLS-оценка $\hat{w}$ является **BLUE**:

- **B**est — имеет минимальную дисперсию
- **L**inear — среди линейных по y оценок
- **U**nbiased — несмещённых
- **E**stimator — оцениватель

Типичные вопросы/ловушки

- Что делать при вырожденности $X^\top X$? (псевдообратная, регуляризация, убрать коллинеарность).
- Почему иногда не добавляют свободный член в регуляризации? (см. билет 12).

11 Билет 11. Градиентный спуск для линейной регрессии

Почему не всегда используют матричное решение

- Вычисление $(X^\top X)^{-1}$ стоит примерно $O(d^3)$ и требует памяти $O(d^2)$.
- Для больших d (много признаков) или для разреженных/потоковых данных это неудобно.
- В реальных задачах часто используется регуляризация и/или нелинейные модели $\Rightarrow$ всё равно нужен итеративный оптимизатор.

Шаг градиентного спуска

$$w_{t+1} = w_t - \eta \nabla_w \mathcal{L}(w_t),$$

где η — шаг обучения (learning rate).

Для MSE: $\nabla_w \mathcal{L}(w) = \frac{2}{n} X^\top (Xw - y)$.

Стохастический градиентный спуск (SGD)

Вместо полного градиента по всем n объектам берём мини-батч B:

$$\nabla_w \mathcal{L}_B(w) = \frac{2}{|B|} X_B^\top (X_B w - y_B).$$

Плюсы SGD

- Дешевле один шаг: $O(|B|d)$ вместо $O(nd)$.
- Можно обучаться на потоковых данных, лучше масштабируется.
- Стохастичность иногда помогает выйти из «плохих» областей в невыпуклых задачах.

Типичные практические приёмы

- Нормализация признаков ускоряет сходимость.
- Подбор learning rate, scheduler, momentum.
- Early stopping по валидации.

12 Билет 12. Регуляризация в линейных моделях: Ridge/Lasso/E Net

Зачем нужна регуляризация

- Борьба с переобучением (уменьшаем сложность/вариативность решения).
- Улучшение обусловленности задачи (особенно при коллинеарности признаков).
- Встраивание априорных предположений: «веса должны быть маленькими» или «много нулей».

Ridge (L2) регуляризация

Задача:

$$\min_w \frac{1}{n} \|Xw - y\|_2^2 + \lambda \|w\|_2^2 .$$

Аналитическое решение:

$$\hat{w}_{\text{ridge}} = (X^\top X + n\lambda I)^{-1} X^\top y.$$

Интуиция: «стягиваем» веса к нулю, уменьшаем дисперсию оценок.

Lasso (L1) регуляризация

$$\min_w \frac{1}{n} \|Xw - y\|_2^2 + \lambda \|w\|_1 .$$

Свойство: может давать **разреженные** решения (много $w_j = 0$) $\Rightarrow$ implicit feature selection. Но задача невдифференцируема в нуле, часто решается координатным спуском.

Elastic Net

Комбинация:

$$\min_w \frac{1}{n} \|Xw - y\|_2^2 + \lambda_1 \|w\|_1 + \lambda_2 \|w\|_2^2 .$$

Полезно, когда признаки коррелированы: L1 выбирает «группы» нестабильно, L2 стабилизирует.

Вероятностная интерпретация (*)

- Ridge $\leftrightarrow$ MAP при гауссовском априоре $w \sim \mathcal{N}(0, \tau^2 I)$.
- Lasso $\leftrightarrow$ MAP при лапласовском априоре (double-exponential) на w.

Свойства и нюансы

- λ — гиперпараметр (подбирается по валидации/CV).
- В sklearn обычно **не регуляризуют bias/intercept** (сдвиг), потому что он отвечает за базовый уровень и не влияет на «сложность» так же, как веса (и зависит от масштаба признаков).
- Регуляризация не единственный способ ограничить решение: ранняя остановка, уменьшение числа признаков, data augmentation и др.

13 Билет 13. Метрики регрессии: MSE, MAE, RMSE, MAPE, SMAPE, R^2

Пусть истинные значения y_i, предсказания $\hat{y}_i$, $i = 1..n$.

MSE и RMSE

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2, \qquad \text{RMSE} = \sqrt{\text{MSE}}.$$

RMSE в тех же единицах, что и y, но сохраняет «квадратичное» наказание больших ошибок.

MAE

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|.$$

Робастнее к выбросам (линейное наказание).

MAPE и SMAPE

$$\mathrm{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n} \left|\frac{y_i - \hat{y}_i}{y_i}\right|.$$

Проблема: плохо определена при $y_i \approx 0$.

$$\mathrm{SMAPE} = \frac{100\%}{n}\sum_{i=1}^{n} \frac{2|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}.$$

Симметричная версия, меньше страдает от нулей, но тоже имеет нюансы.

R^2 (коэффициент детерминации)

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \qquad \bar{y} = \frac{1}{n}\sum_i y_i.$$

Интуиция: насколько модель лучше константного предсказания $\bar{y}$ по MSE. R^2 может быть отрицательным.

Когда что использовать

- MSE/RMSE: когда большие ошибки крайне нежелательны.
- MAE: когда важна робастность.
- MAPE/SMAPE: когда важна относительная ошибка (например, спрос), но осторожно с нулями.
- R^2: удобно для отчётов/сравнения с бейзлайном, но не всегда соответствует бизнес-ценности.

14 Билет 14. Параметры и гиперпараметры

Определения

- **Параметры** — то, что *обучается* из данных оптимизацией: w, b в линейной модели, веса нейросети, пороги в деревьях.
- **Гиперпараметры** — то, что *задаётся извне* и управляет обучением/мощностью модели: λ в регуляризации, глубина дерева, число деревьев, learning rate.

Свойства

- Параметров обычно много (особенно в DL), гиперпараметров — существенно меньше.
- Параметры минимизируют training loss, гиперпараметры выбирают по валидации/кросс-валидации.
- Гиперпараметры управляют компромиссом bias–variance и вычислительной стоимостью.

Примеры

- Линейная регрессия: параметры w, b, гиперпараметр λ (ridge).
- Дерево решений: параметры — структура дерева и пороги, гиперпараметры — max_depth, min_samples_leaf, критерий (gini/entropy).

Практический совет

Если модель переобучается, часто лечат гиперпараметрами сложности (регуляризация, ограничение глубины, ранняя остановка) и/или данными (больше данных, лучше разметка).

15 Билет 15. Валидация и оценка модели: сплиты, CV, утечки

Underfitting и overfitting

- **Недообучение (underfitting)**: модель слишком простая, высокие ошибки и на train, и на val.
- **Переобучение (overfitting)**: на train хорошо, на val плохо; модель запомнила шум/-частности.
- Диагностика: кривые обучения, сравнение метрик train/val, устойчивость к CV.

Что такое «хороший сплит»

Идеально: train и test из одного распределения и независимы. Практически: нужно имитировать будущую эксплуатацию и избегать зависимости объектов.

Основные типы разбиений (4 популярных)

- **Случайный (random) сплит** — базовый вариант.
- **Стратифицированный** — сохраняет доли классов (важно при дисбалансе).
- **По группам (group split)** — когда есть группы/пользователи/сессии, которые нельзя «разрывать».
- **Временной (time-based)** — train раньше, test позже (временные ряды, логовые данные).

Train / validation / test

- **Train**: оптимизируем параметры.
- **Validation**: выбираем гиперпараметры/модель, делаем early stopping.
- **Test**: финальная честная оценка (использовать один раз в конце).

Кросс-валидация

- Разбиваем данные на K фолдов, по очереди держим один фолд как val.
- Плюс: более стабильная оценка качества.
- Минус: дороже по вычислениям; нужно аккуратно делать весь препроцессинг внутри каждого фолда.

Утечки данных (data leakage)

Примеры утечек:

- Признаки из будущего (например, «количество оплат за следующие 7 дней»).
- Препроцессинг на всех данных до сплита (scaler, PCA, таргет-энкодинг без CV).
- Дубликаты/похожие объекты попали и в train, и в test (по пользователю/товару/серии).

Правило: всё, что «учится» (статистика, словарь, PCA), должно фититься **только на train**.

16 Билет 16. Линейная классификация

Постановка задачи

Классификация: по $x \in \mathbb{R}^d$ предсказать класс y (дискретная метка). Бинарный случай: $y \in \{-1, +1\}$ или $y \in \{0, 1\}$.

Варианты меток

- «Жёсткие» метки: $0/1$ или $-1/+1$.
- Вероятности/soft labels: $y \in [0, 1]$ (например, после агрегации разметки).
- Мультикласс: $y \in \{1, \ldots, K\}$; multi-label: несколько меток сразу.

Линейный классификатор и геометрия

Скор (score):

$$s(x) = w^\top x + b.$$

Решающее правило (бинарное):

$$\hat{y} = \text{sign}(s(x)).$$

Граница решений: гиперплоскость $w^\top x + b = 0$.

Маржа (margin)

Геометрическая маржа точки:

$$\gamma_i = \frac{y_i(w^\top x_i + b)}{\|w\|_2}.$$

Интуиция: насколько уверенно и далеко объект от границы в правильную сторону. Большая маржа $\Rightarrow$ обычно лучше обобщение (см. SVM).

Почему много разных функций потерь

- 0–1 loss недифференцируема.
- Используют суррогаты: hinge (SVM), logistic (логрег), exponential (AdaBoost).
- Разные потери дают разные свойства: устойчивость, калибровка вероятностей, оптимизационная удобность.

17 Билет 17. Логистическая регрессия

Основная идея и геометрия

Логистическая регрессия — линейная модель по признакам, которая выдаёт **вероятность** класса:

$$p(y = 1 \mid x) = \sigma(w^\top x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Граница решений при пороге 0.5 совпадает с гиперплоскостью $w^\top x + b = 0$.

Сигмоида как вероятность

Свойства:

- $\sigma(z) \in (0, 1)$, монотонно возрастает.
- $\sigma(0) = 0.5$, $\sigma(-z) = 1 - \sigma(z)$.

Логистическая потеря (logloss)

Для $y \in \{0, 1\}$:

$$\ell(\hat{p}, y) = -\big(y \log \hat{p} + (1 - y) \log(1 - \hat{p})\big), \quad \hat{p} = \sigma(w^\top x + b).$$

Эквивалентно **negative log-likelihood** Бернулли (MLE-вывод*).

Связь с MLE (*)

Если $y_i \mid x_i \sim \text{Bernoulli}(\sigma(w^\top x_i + b))$, то MLE по (w, b) даёт минимизацию суммы logloss.

Калибровка вероятностей

Логистическая регрессия часто даёт неплохую калибровку «из коробки» (в отличие от некоторых моделей), но при сильном дисбалансе/смещениях может требоваться calibration (Platt scaling, isotonic).

GLM (*)

Логрег — частный случай **обобщённых линейных моделей**: линейный предиктор $\eta = w^\top x +$ link-функция (логит) связывает η с параметром распределения.

18 Билет 18. Стратегии мультиклассовой классификации: OvR и OvO

Пусть K классов.

One-vs-Rest (OvR, One-vs-All)

- Обучаем K бинарных классификаторов: «класс k против остальных».
- На предсказании берём класс с максимальным score/вероятностью.
- Плюсы: просто, линейно по K.
- Минусы: классы «остальные» сильно неоднородны; вероятности могут быть некалиброваны.

One-vs-One (OvO)

- Обучаем $\frac{K(K-1)}{2}$ бинарных моделей для каждой пары классов.
- Предсказание через голосование или агрегацию вероятностей.
- Плюсы: каждая задача проще (только два класса), иногда выше качество.
- Минусы: много моделей, дороже по памяти/времени (особенно при большом K).

Альтернатива

- Нативная мультиклассовая модель: softmax-регрессия (см. билет 39).
- Деревья/бустинг обычно «из коробки» поддерживают мультикласс.

19 Билет 19. Метрики классификации

Метрика в ML

Метрика качества — функция, оценивающая качество модели на данных. Отличие от loss: loss оптимизируется при обучении (должна быть удобна), метрика может быть недифференцируема и ближе к бизнес-цели.

Confusion matrix (бинарная)

Для positive класса:

- TP — истинно-положительные, FP — ложно-положительные,
- TN — истинно-отрицательные, FN — ложно-отрицательные.

Правило присвоения метки обычно: $\hat{y} = 1$ если $\hat{p} \geq t$ (порог t).

Accuracy и balanced accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

При дисбалансе accuracy может быть высокой у «глупой» модели. Balanced accuracy:

$$\text{BAcc} = \frac{1}{2}\left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP}\right).$$

Precision, Recall, F1

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \qquad F1 = \frac{2PR}{P + R}.$$

Интуиция: precision — «точность положительных предсказаний», recall — «полнота нахождения положительных».

(*) Связанные термины: sensitivity=recall, specificity=$\frac{TN}{TN+FP}$.

Дисбаланс и разная стоимость ошибок

Если positive редок (например 5%), то:

- меняют метрики (PR-AUC, F1, balanced accuracy),

- меняют порог,
- используют взвешивание классов, oversampling/undersampling,
- формулируют cost-sensitive loss.

ROC и ROC-AUC

ROC-кривая: TPR (=Recall) по оси y и FPR=$\frac{FP}{FP+TN}$ по оси x при изменении порога. ROC-AUC — площадь под ROC; интерпретация: вероятность, что случайный positive получит скор выше случайного negative.

PR-AUC

Precision-Recall кривая полезнее при сильном дисбалансе, потому что precision напрямую зависит от доли FP.

Мультикласс и усреднения

- **Macro** — среднее метрики по классам (важно для редких классов).
- **Micro** — считаем суммарные TP/FP/FN по всем классам (важно для общей доли объектов).

20 Билет 20. Нормы матриц (*)

Норма вектора p-норма

$$\|x\|_p = \left(\sum_{i=1}^{d} |x_i|^p\right)^{1/p}, \quad p \geq 1.$$

Важные случаи: $p = 1, 2, \infty$.

Нормы матриц (самые частые)

- **Фробениусова норма**:

$$\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2} = \sqrt{\text{trace}(A^\top A)}.$$

- **Спектральная (операторная) 2-норма**:

$$\|A\|_2 = \max_{\|x\|_2=1} \|Ax\|_2 = \sigma_{\max}(A),$$

  где $\sigma_{\max}$ — максимальное сингулярное значение.
- **Индуктивные нормы** $\|A\|_p = \max_{\|x\|_p=1} \|Ax\|_p$.

Зачем это в ML

- Регуляризация, оценки устойчивости, Lipschitz-константы.
- PCA/SVD формулировки через минимизацию норм.

21 Билет 21. Support Vector Machine (SVM)

Геометрическая интерпретация

SVM ищет разделяющую гиперплоскость с **максимальной маржой**. Для линейно разделимого случая хотим максимизировать минимальную маржу точек.

Линейно разделимый случай: primal (*)

При $y_i \in \{-1, +1\}$:

$$\min_{w,b} \ \frac{1}{2} \|w\|_2^2 \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1, \ \forall i.$$

Минимизируем $\|w\|^2 \Leftrightarrow$ максимизируем маржу $1/\|w\|$.

Dual (*)

Через множители Лагранжа получаем задачу по $\alpha_i \geq 0$:

$$\max_{\alpha} \ \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle, \quad \text{s.t.} \ \sum_i \alpha_i y_i = 0.$$

Точки с $\alpha_i > 0$ — **опорные векторы**.

Неразделимый случай: slack (*)

Добавляем $\xi_i \geq 0$:

$$\min_{w,b,\xi} \ \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i.$$

Hinge loss

Эквивалентная (по смыслу) форма:

$$\ell_{\text{hinge}}(y, s) = \max(0, 1 - ys), \quad s = w^\top x + b.$$

SVM минимизирует hinge loss + L2-регуляризацию.

Kernel trick

В dual все данные входят только через скалярные произведения $\langle x_i, x_j \rangle$. Заменяем на ядро $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$ без явного ϕ. Свойства ядра: симметричность и положительная полуопределённость (матрица Грама PSD).

Примеры ядер: RBF (гауссово), полиномиальное, линейное.

Когда SVM хорош

Средние размеры данных, хороший «зазор», высокое качество на табличных данных; но плохо масштабируется на очень большие n без специальных методов.

22 Билет 22. Principal Component Analysis (PCA)

Постановка задачи

Дано $X \in \mathbb{R}^{n \times d}$ (обычно центрированная матрица признаков). PCA ищет ортонормированный базис, в котором первые компоненты объясняют максимум дисперсии.

Эквивалентно: найти направление v с $\|v\| = 1$, максимизирующее

$$\mathrm{Var}(Xv) = \frac{1}{n} \|Xv\|_2^2 .$$

Для k компонент ищем матрицу $V \in \mathbb{R}^{d \times k}$ с $V^\top V = I$.

Связь с SVD

Если X центрирована, то SVD:

$$X = U \Sigma V^\top .$$

Столбцы V — главные направления (principal axes), $\Sigma^2/(n-1)$ связаны с дисперсиями компонент.

Теорема Эккарта–Юнга (*)

Лучшее приближение матрицы ранга k в норме Фробениуса даётся усечённой SVD:

$$X_k = U_{:,1:k} \Sigma_{1:k,1:k} V^\top_{:,1:k}.$$

Итеративные формулировки (2 идеи)

- **Power iteration**: для первой компоненты итеративно $v \leftarrow X^\top X v$ с нормировкой.
- **Oja's rule**: стохастическая версия обновления для потоковых данных.

Препроцессинг для PCA

- Центрирование (обязательно): $X \leftarrow X - \bar{X}$.
- Масштабирование (часто нужно): если признаки в разных единицах, используйте StandardScaler.
- Работа с пропусками: либо импутация, либо методы, учитывающие missing.

Интерпретация

- PCA — линейное снижение размерности, сохраняет максимум дисперсии.
- Компоненты ортогональны; но «макс дисперсия» не всегда = «макс полезной информации для таргета».

23 Билет 23. k-ближайших соседей (kNN) для классификации и регрессии

Идея алгоритма

Для объекта x ищем k ближайших соседей в обучающей выборке по метрике расстояния $d(x, x_i)$.

- **Классификация**: класс по большинству голосов (или взвешенное голосование).
- **Регрессия**: среднее/медиана значений y у соседей (или взвешенное среднее).

Гиперпараметры (и как влияют)

- k: маленький $k \Rightarrow$ низкий bias, высокая variance; большой k сглаживает.
- Метрика: Euclidean, Manhattan, cosine (особенно для текстов/эмбеддингов).
- Взвешивание соседей: например $w_i = \frac{1}{d(x,x_i)+\epsilon}$.
- Способ обработки категориальных признаков (обычно их нужно кодировать).

Почему важен масштаб признаков

Расстояния чувствительны к масштабу: признак с большим диапазоном доминирует. Решение: нормализация/стандартизация (StandardScaler), или метрика, учитывающая масштаб.

Сложность

- Наивный поиск: $O(nd)$ на один запрос.
- Память: хранится весь train (lazy learner).

Когда работает хорошо

Низкая/средняя размерность, хорошие метрики близости, много данных, плавная зависимость.

24 Билет 24. Индексы для kNN: идеи, HNSW, векторные БД

Зачем нужны индексы

Наивный поиск ближайших соседей $O(n)$ на запрос становится дорогим при большом n. Индексы ускоряют поиск (часто *приближённый* ANN) ценой памяти/приближённости.

Идеи индексов (минимум 3)

- **k-d tree** / **ball tree**: пространственные деревья (хорошо в малой размерности).
- **VP-tree**: разбиение по расстоянию до «опорной точки».
- **LSH (Locality Sensitive Hashing)**: хеши, сохраняющие близость (часто для косинуса).
- **Графовые индексы**: строим граф «близости» и ищем по нему (HNSW, NSG).
- **IVF+PQ**: кластеризация + продукт-квантизация (как в FAISS) для очень больших коллекций.

HNSW (Hierarchical Navigable Small World)

Идея: многослойный граф, где верхние слои разрежены и помогают быстро найти область поиска.

- (*) Построение: добавляем точки по одной, соединяем с ближайшими в текущем графе, контролируем степень.

- Использование: начиная с верхнего слоя, жадно двигаемся к более близким вершинам, спускаемся вниз, на нижнем слое делаем локальный поиск.
- Гиперпараметры: `M` (степень), `efConstruction`, `efSearch`.

Примеры векторных баз данных

FAISS (библиотека), Milvus, Weaviate, Pinecone и др. (инструменты для ANN-поиска и хранения эмбеддингов).

25 Билет 25. The Bitter Lesson

О чём статья

"The Bitter Lesson" (Rich Sutton) — наблюдение из истории ИИ: методы, которые **эффективно используют вычисления и данные**, в долгосрочной перспективе выигрывают у методов с «ручными» знаниями и сложными эвристиками.

Главные выводы

- За десятилетия прогресса выигрывали подходы, масштабируемые по compute (поиск, обучение на больших данных, градиентные методы).
- Ручные доменные знания часто дают краткосрочные улучшения, но плохо масштабируются и ограничивают потолок.
- Поэтому стоит вкладываться в: данные, вычисления, общие методы оптимизации, архитектуры, которые умеют учиться.

Как применить в ML-инженерии

- Хороший пайплайн данных и качественная разметка иногда важнее «крутой модели».
- Бейзлайн + масштабирование (данных/модели/compute) часто дают больше, чем тонкие хендкрафт-фичи.
- Но в бизнесе ограничения (латентность/интерпретируемость/стоимость) могут сделать простую модель предпочтительнее.

26 Билет 26. Деревья решений

Жадные алгоритмы

Жадный (greedy) алгоритм делает локально оптимальный выбор на каждом шаге, не гарантируя глобального оптимума.

Как строится дерево

1. В корне есть все объекты train.
2. Выбираем признак и порог, максимизирующие улучшение критерия (информационный выигрыш).
3. Делим выборку на левую/правую ветви.
4. Повторяем рекурсивно до условий остановки.

Критерии разбиения

Для классификации:

- **Энтропия**: $H(p) = -\sum_c p_c \log p_c$.
- **Gini impurity**: $G(p) = 1 - \sum_c p_c^2$.
- Разбиение выбирают по максимальному снижению неоднородности (information gain).

Для регрессии часто минимизируют MSE внутри листьев.

Ответ в листе

- Классификация: обычно **модальный класс** в листе или вероятности классов по частотам.
- Регрессия: среднее/медиана y в листе.

Пропуски, категориальные и бинаризация

- Пропуски: отдельная ветка/направление по умолчанию, либо импутация.
- Категориальные: one-hot, либо специальные реализации (например, CatBoost).
- Бинаризация: разбиение по порогу для числовых признаков.

Переобучение и pruning (*)

Глубокое дерево может добиться нулевой ошибки на train (если объекты уникальны). Pruning: обрезка ветвей по валидации или через cost-complexity.

Основные гиперпараметры (4+)

- `max_depth`, `min_samples_split`, `min_samples_leaf`,
- `max_features`, `criterion`, `max_leaf_nodes`.

27 Билет 27. Bias–variance–noise decomposition

Откуда берётся случайность в ошибке

Даже при фиксированном x истинный y может содержать шум (непредсказуемую компоненту). Кроме того, модель зависит от обучающей выборки $\mathcal{D}$.

Классическое разложение для MSE

Для регрессии с квадратом ошибки:

$$\mathbb{E}_{\mathcal{D},y}\left[(\hat{f}_{\mathcal{D}}(x) - y)^2\right] = \underbrace{\left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)] - f^\star(x)\right)^2}_{\text{Bias}^2} + \underbrace{\mathrm{Var}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}(x))}_{\text{Variance}} + \underbrace{\mathrm{Var}(y \mid x)}_{\text{Noise}}.$$

Интуиция:

- Bias — систематическая ошибка (слишком простая модель).
- Variance — чувствительность к данным (слишком сложная/неустойчивая модель).
- Noise — неизбежная ошибка из-за шума/неполноты признаков.

Bias–variance tradeoff

Увеличение сложности модели обычно уменьшает bias, но увеличивает variance. Регуляризация и ансамбли (bagging) снижают variance; бустинг часто снижает bias.

Beyond MSE (*)

Для других потерь разложение сложнее; но интуиция про компромисс bias/variance сохраняется.

Double descent (*)

В некоторых режимах (особенно при большой параметризации) ошибка как функция сложности может иметь второй спад: после точки интерполяции качество снова улучшается.

28 Билет 28. Бэггинг, бутстрэп и Random Forest

Мотивация через bias–variance

Среднее по нескольким моделям снижает variance (если ошибки частично независимы).

Bootstrap

Бутстрэп-выборка размера n строится выбором n объектов **с возвращением** из исходного train. В среднем около 63.2% уникальных объектов попадает в бутстрэп (остальные — out-of-bag).

Bagging

1. Сэмплируем B бутстрэп-выборок.
2. Обучаем базовую модель на каждой.
3. Агрегируем предсказания (среднее для регрессии, голосование для классификации).

Random subspace

Дополнительно к бутстрэпу случайно выбираем подмножество признаков, чтобы декоррелировать модели.

Random Forest

Ансамбль деревьев с:

- обучением каждого дерева на бутстрэпе,
- выбором случайного поднабора признаков в каждой вершине.

Плюсы: сильный универсальный бейзлайн, устойчивость. Минусы: большие модели, менее интерпретируемы, чем одно дерево.

OOB-оценка

Out-of-bag объекты можно использовать для оценки качества без отдельного validation.

29 Билет 29. Обнаружение аномалий (*)

Постановка

Аномалия — объект, который сильно отличается от «нормы». Часто нет разметки аномалий $\Rightarrow$ используются unsupervised/one-class методы.

Isolation Forest

- Идея: аномалии легче «изолировать» случайными разбиениями.
- Строим много случайных деревьев разбиения признаков; оцениваем среднюю длину пути до листа.
- Чем короче путь, тем более вероятно, что объект аномален.

One-Class SVM

- Обучаем границу, отделяющую «нормальные» точки от начала координат в feature space.
- Часто используют RBF-ядро; параметры ν (доля выбросов) и γ.
- Минус: чувствителен к масштабу признаков и плохо масштабируется на большие n.

Практические нюансы

- Важно определить, что именно считается аномалией (редкость? риск? ошибка измерения?).
- Метрики качества без разметки сложны: часто используют ручную проверку top-k.

30 Билет 30. Градиентный бустинг

Идея

Бустинг строит аддитивную модель:

$$F_M(x) = \sum_{m=1}^{M} \gamma_m h_m(x),$$

где h_m — слабые модели (обычно небольшие деревья), а M — число итераций.

Процедура для MSE

Для квадратичной потери на итерации m:

1. Текущие предсказания: $\hat{y}^{(m-1)} = F_{m-1}(x)$.
2. Остатки (антиградиент): $r_i = y_i - \hat{y}_i^{(m-1)}$.
3. Обучаем новое дерево h_m на парах (x_i, r_i).
4. Обновляем: $F_m(x) = F_{m-1}(x) + \eta h_m(x)$.

Здесь η — learning rate.

Бустинг как градиентный спуск

В общем случае (любая дифференцируемая потеря) учим h_m приближать **антиградиент** по предсказаниям:

$$r_i = - \left. \frac{\partial \ell(y_i, F(x_i))}{\partial F} \right|_{F=F_{m-1}}.$$

Это «градиентный спуск в пространстве функций».

Выбор базовой модели

Обычно используют деревья небольшой глубины (stumps/малые деревья), чтобы каждое добавление было простым, но ансамбль становился сильным.

Параллелизм

Итерации бустинга последовательны (каждое дерево зависит от предыдущих), но внутри итерации можно параллелить поиск лучших сплитов, построение гистограмм, обработку признаков.

31 Билет 31. Библиотеки градиентного бустинга: XGBoost, LightGBM, CatBoost

XGBoost

- Плюсы: очень зрелая библиотека, много возможностей (регуляризация, разные задачи), стабильность.
- Минусы: может быть тяжелее на очень больших датасетах по сравнению с LGBM.

LightGBM

- Плюсы: быстрый на больших данных, эффективные гистограммы, хорошая поддержка категориальных (частично).
- Минусы: иногда чувствителен к настройкам; риск переобучения при неправильных параметрах (очень гибкий).

CatBoost

- Плюсы: сильная работа с категориальными признаками (таргет-статистики с борьбой с утечкой), часто «из коробки» даёт хороший результат.
- Минусы: может быть медленнее/тяжелее; сложнее интерпретировать внутренности.

Общее

- Все три — мощные бейзлайны для табличных данных.
- Важно уметь назвать ключевые гиперпараметры: `n_estimators`, `learning_rate`, `max_depth`, `subsample`, `colsample`.

32 Билет 32. Важности признаков и SHAP

Зачем нужны важности

- Понять, какие признаки влияют на предсказание.
- Найти утечки или «подозрительные» признаки.
- Объяснить модель бизнесу (с оговорками).

Shapley Additive Explanations

SHAP основывается на **значениях Шепли** из теории кооперативных игр: каждый признак — «игрок», предсказание — «выигрыш».

Идея: вклад признака j — средний маржинальный вклад по всем порядкам добавления признаков.

Аксиомы (*)

Классические свойства значений Шепли:

- **Efficiency**: сумма вкладов равна разнице между предсказанием и базовой линией.
- **Symmetry**: одинаковые признаки получают одинаковый вклад.
- **Dummy**: бесполезный признак имеет нулевой вклад.
- **Additivity**: для суммы игр вклад — сумма вкладов.

SHAP значения в ML

- **Локальные объяснения**: для конкретного объекта x получаем вклады признаков.
- **Глобальные важности**: агрегируем $|\text{SHAP}_j|$ по датасету.
- Для деревьев есть быстрые алгоритмы (TreeSHAP).

Нюансы

- SHAP объясняет модель, а не «истинную причинность».
- Коррелированные признаки делят вклад неоднозначно (зависит от определения «отсутствия признака»).

33 Билет 33. Оптимизация гиперпараметров (Hyperparameter Optimization)

Класс задач

Хотим найти:

$$\lambda^\star = \arg\min_{\lambda \in \Lambda} \ \text{ValMetric}(\lambda),$$

где λ — гиперпараметры, а ValMetric — качество на валидации/CV.

Почему это сложно

- Функция обычно дорогая (нужно обучить модель).
- Может быть шумной (случайность в обучении/сплите).
- Часто нет градиентов по гиперпараметрам (чёрный ящик).

Основные подходы

- **Grid search**: перебор по сетке (плохо масштабируется).
- **Random search**: случайный поиск (часто лучше грида при ограниченном бюджете).
- **Bayesian optimization**: строим surrogate (например, GP/TPE) и выбираем следующую точку.
- **Multi-fidelity**: Hyperband/ASHA — ранняя остановка плохих конфигураций.
- Эволюционные/Population-based методы (PBT).

Библиотеки

Optuna, Hyperopt, scikit-optimize, Ray Tune и др.

Практические советы

- Делать хорошую постановку сплита и контроль утечек *до* тюнинга.
- Фиксировать seed для сравнимости, но проверять устойчивость.
- Начинать с разумных диапазонов (в лог-шкале для learning rate, λ).

34 Билет 34. Матричный анализ: производные (минимум)

Скалярное произведение двух векторов

Пусть $a, b \in \mathbb{R}^d$.

$$f(a) = a^\top b = \sum_{i=1}^{d} a_i b_i \quad \Rightarrow \quad \frac{\partial f}{\partial a} = b, \quad \frac{\partial f}{\partial b} = a.$$

Квадратичная форма

$$f(a) = a^\top a = \|a\|_2^2 \quad \Rightarrow \quad \frac{\partial f}{\partial a} = 2a.$$

Матрица и вектор

Пусть $A \in \mathbb{R}^{m \times d}$, $x \in \mathbb{R}^d$, $y = Ax \in \mathbb{R}^m$. Если $f(x) = c^\top (Ax)$ для $c \in \mathbb{R}^m$, то:

$$f(x) = (A^\top c)^\top x \;\Rightarrow\; \frac{\partial f}{\partial x} = A^\top c.$$

Практическая форма (vector-Jacobian product)

В backprop обычно распространяют градиент $g = \frac{\partial \mathcal{L}}{\partial y} \in \mathbb{R}^m$:

$$\frac{\partial \mathcal{L}}{\partial x} = A^\top g, \qquad \frac{\partial \mathcal{L}}{\partial A} = g x^\top.$$

Это «минимальный набор» для понимания линейного слоя и backprop.

35 Билет 35. Концепция нейронной сети

Зачем нужны нейросети

Нейросети — универсальные аппроксиматоры сложных нелинейных зависимостей, которые обучаются градиентными методами и хорошо масштабируются на больших данных/compute.

Проблема XOR

XOR неразделим линейно в 2D: один линейный классификатор не может решить задачу. Небольшая сеть с нелинейностью (2 слоя) решает XOR, что мотивирует добавлять **скрытые слои** и **активации**.

Определение слоя

Слой — параметризованное преобразование $h^{(l)} = \phi(W^{(l)}h^{(l-1)} + b^{(l)})$. Сеть — композиция слоёв.

Графическая нотация

Нейросеть можно рассматривать как вычислительный граф: вершины — операции/тензоры, рёбра — зависимости. Backprop — применение цепного правила по графу.

Эмбеддинг

Embedding — обучаемое отображение дискретных объектов (слово, товар, пользователь) в вектор:

$$\text{token } t \mapsto e_t \in \mathbb{R}^d.$$

Используется в NLP, рекомендациях, графах. По сути это таблица параметров размера (число токенов) $\times d$.

36 Билет 36. Линейный слой

Как ещё называют

Linear layer = **полносвязный слой** (fully-connected, FC) = **dense** = **affine** преобразование.

Формула

$$y = Wx + b,$$

где $W \in \mathbb{R}^{m\times d}$, $b \in \mathbb{R}^m$, вход $x \in \mathbb{R}^d$, выход $y \in \mathbb{R}^m$.

Гиперпараметры

- Размерности: d (in_features), m (out_features).
- Наличие bias (иногда отключают).

Сколько параметров

$$\#\text{params} = m \cdot d + m \quad \text{(если есть bias)}.$$

Например, слой $10 \to 3$ имеет $3 \cdot 10 + 3 = 33$ параметра.

Инициализация

- Важно избегать взрыва/затухания сигналов и градиентов.
- Xavier/Glorot — для tanh/sigmoid; He/Kaiming — для ReLU-подобных.

Универсальная теорема аппроксимации

Сеть с хотя бы одним скрытым слоем и нелинейностью может аппроксимировать любую непрерывную функцию на компактном множестве (при достаточном числе нейронов). Это *существование*, а не гарантия лёгкости обучения.

37 Билет 37. Функции активации

Зачем нужны

Без нелинейностей композиция линейных слоёв остаётся линейной $\Rightarrow$ модель невыразительна. Активации вводят нелинейность и позволяют учить сложные функции.

Примеры (4+)

- **Sigmoid**: $\sigma(z) = \frac{1}{1+e^{-z}}$.
- **tanh**: $\tanh(z)$ (центрирована около 0).
- **ReLU**: $\max(0, z)$.
- **Leaky ReLU**: $\max(\alpha z, z)$.
- **GELU**: $z\Phi(z)$ (часто в трансформерах).
- **Softplus**: $\log(1 + e^z)$.

Критерии сравнения (что рассказать)

- Диапазон значений (ограниченная/неограниченная).
- Насыщение (vanishing gradients у sigmoid/tanh на больших $|z|$).
- Наличие «мертвых» нейронов (ReLU при постоянных отрицательных входах).
- Гладкость/дифференцируемость (важно для оптимизации).
- Вычислительная простота.

Практические правила

- ReLU/варианты — дефолт для MLP/CNN.
- GELU/SiLU — популярны в современных архитектурах.
- Sigmoid часто используют на выходе для бинарной вероятности, softmax — для мультикласса.

38 Билет 38. Backpropagation

Цепное правило

Если $z = f(x)$ и $L = g(z)$, то:

$$\frac{dL}{dx} = \frac{dL}{dz} \cdot \frac{dz}{dx}.$$

В многомерном случае используют якобианы; на практике считают **vector-Jacobian product**.

Идея backprop

- Прямой проход: считаем значения во всех узлах графа.
- Обратный проход: распространяем градиенты от loss к параметрам.
- Сложность обратного прохода обычно сравнима с прямым (константный множитель).

Пример для линейного слоя

$y = Wx + b$, $g = \frac{\partial L}{\partial y}$:

$$\frac{\partial L}{\partial x} = W^\top g, \qquad \frac{\partial L}{\partial W} = g x^\top, \qquad \frac{\partial L}{\partial b} = g.$$

Практические нюансы

- Нужно хранить активации прямого прохода для вычисления градиентов.
- Для больших моделей важны экономия памяти (checkpointing) и численная стабильность.

39 Билет 39. Мультиклассовая классификация: Softmax и Cross-Entropy

Softmax

Для логитов $z \in \mathbb{R}^K$:

$$p_k = \mathrm{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}.$$

Свойства:

- $p_k \in (0, 1)$ и $\sum_k p_k = 1$.
- Инвариантность к сдвигу: $\mathrm{softmax}(z) = \mathrm{softmax}(z + c)$.

Численная стабильность (*)

Используют трюк:

$$p_k = \frac{e^{z_k - \max_j z_j}}{\sum_j e^{z_j - \max_j z_j}},$$

чтобы избежать переполнения экспоненты.

Cross-entropy loss

Если истинная метка one-hot y:

$$\ell(p, y) = -\sum_{k=1}^{K} y_k \log p_k.$$

В частном случае правильного класса c: $\ell = -\log p_c$. Это NLL для категориального распределения.

Информация и энтропия

- Информационное содержание события x: $I(x) = -\log p(x)$.
- Энтропия: $H(p) = -\sum_x p(x) \log p(x)$.

Perplexity

Для языковых моделей:

$$\text{PPL} = \exp\left(\frac{1}{n}\sum_{i=1}^{n} -\log p(x_i)\right) = \exp(\text{cross-entropy}).$$

Меньше — лучше.

KL-дивергенция

$$D_{\text{KL}}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}.$$

Связь: cross-entropy $H(p, q) = H(p) + D_{\text{KL}}(p\|q)$.

40 Билет 40. Компьютерное железо для ML

Архитектура фон Неймана

- Программа и данные хранятся в памяти, процессор выполняет инструкции последовательно.
- Узкое место: **пропускная способность памяти** (von Neumann bottleneck).

Локальность данных

- Важно, чтобы данные лежали «рядом» в памяти (cache-friendly), иначе CPU/GPU ждут память.
- В ML часто ускорение достигается оптимизацией доступа к тензорам и батчированием.

Поток данных в PyTorch (идея)

Тензоры живут либо на CPU, либо на GPU. Важно минимизировать копирования CPU↔GPU и делать вычисления батчами.

Таксономия Флинна

Классификация по потокам команд/данных: SISD, SIMD, MISD, MIMD. GPU близок к SIMD/SIMT подходу (много потоков над одинаковыми инструкциями).

CPU vs GPU: ключевые отличия

- CPU: меньше ядер, сильные ядра, сложная логика, хорош для разветвлений.
- GPU: очень много простых ядер, огромная параллельность, идеально для матричных операций.

CUDA

NVIDIA CUDA — платформа/модель программирования для вычислений на GPU NVIDIA. В ML CUDA важна как «низкий уровень» для ускоренных операций (cuDNN, cuBLAS и др.).

41 Билет 41. PyTorch

Зачем ещё одна матричная библиотека

- Автодифференцирование (autograd) для градиентов.
- GPU/TPU ускорение, смешанная точность, распределённое обучение.
- Удобное описание моделей как классов `nn.Module`.

Бэкенды

CPU, CUDA (GPU), иногда MPS (Apple), и др. (в зависимости от сборки).

Архитектура: что важно знать

- `torch.Tensor` — основной тип данных.
- `autograd` — вычисление градиентов.
- `nn.Module`: базовый класс модели; `nn.Sequential` для простых цепочек.
- `torch.optim`: оптимизаторы (SGD, Adam) и schedulers.
- Losses: `nn.CrossEntropyLoss`, `nn.MSELoss` и др.

Поток данных (dataflow) в обучении

1. `Dataset` описывает, как получать пары (x, y).
2. `DataLoader` батчит и параллелит загрузку.
3. Forward pass: `y_pred = model(x)`.
4. Loss: `loss = criterion(y_pred, y)`.
5. Backward: `loss.backward()`.
6. Step: `optimizer.step()`, затем `optimizer.zero_grad()`.

Режимы train/eval

- `model.train()` включает dropout и обновление BatchNorm статистик.
- `model.eval()` отключает dropout и фиксирует BatchNorm (см. билет 60).

42 Билет 42. Рекуррентные нейросети (RNN)

Последовательные данные и токены

Последовательности: текст, аудио, временные ряды, события. **Токен** — элемент последовательности (слово/символ/событие).

Vanilla RNN: блок и размеры

Пусть $x_t \in \mathbb{R}^d$, скрытое состояние $h_t \in \mathbb{R}^h$:

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b), \qquad \hat{y}_t = g(W_o h_t).$$

$W_x \in \mathbb{R}^{h\times d}$, $W_h \in \mathbb{R}^{h\times h}$.

Кодирование токенов

- One-hot $\to$ embedding-слой.
- Предобученные эмбеддинги или обучаемые с нуля.

Типовые постановки

- Many-to-one: классификация текста по последовательности.
- Many-to-many: теггинг (NER), посимвольная разметка, перевод (encoder-decoder).
- Прогноз временного ряда: next-step prediction.

Backpropagation Through Time (BPTT)

Разворачиваем RNN по времени как глубокую сеть и применяем backprop по всем t. Обычно ограничивают длину (truncated BPTT) для эффективности.

Активации и гиперпараметры

- Внутри RNN часто используют tanh/sigmoid (исторически), сейчас могут быть вариации.
- Гиперпараметры: размер hidden state h, число слоёв, dropout, длина развёртки.
- Число параметров: $h \cdot d + h \cdot h + h$ (на один слой, без выхода).

Проблемы градиентов

- **Vanishing gradients** и **exploding gradients** при длинных зависимостях.
- Решения: gradient clipping, лучшая инициализация, LSTM/GRU (см. билет 43), нормализации.

43 Билет 43. Ячейка LSTM и GRU

Зачем усложнять RNN

Vanilla RNN плохо держит долгие зависимости из-за затухающих/взрывающихся градиентов. LSTM вводит **память** c_t и **ворота** (gates), чтобы контролировать поток информации.

Идея памяти и ворот (LSTM)

Типичные уравнения (сокращённо):

$$\begin{aligned} f_t &= \sigma(W_f[x_t, h_{t-1}] + b_f) && \text{(forget)} \\ i_t &= \sigma(W_i[x_t, h_{t-1}] + b_i) && \text{(input)} \\ \tilde{c}_t &= \tanh(W_c[x_t, h_{t-1}] + b_c) && \text{(candidate)} \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\ o_t &= \sigma(W_o[x_t, h_{t-1}] + b_o) && \text{(output)} \\ h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

Почему помогает: есть почти линейный путь для градиента через c_t (при $f_t \approx 1$).

Гиперпараметры и число параметров

Hidden size h, input size d. В LSTM четыре «набора» матриц: параметров порядка $4 \cdot (h(d + h) + h)$ (грубо, с bias).

GRU

GRU проще LSTM (без отдельной памяти c_t), использует update/reset gates. Часто даёт сопоставимое качество при меньшем числе параметров.

Практика

Сегодня для длинных последовательностей часто предпочитают attention/transformers, но RNN/LSTM важны как базовая модель и для некоторых задач на маленьких данных.

44 Билет 44. Многослойные RNNs

Stacked (multi-layer) RNN

- Несколько рекуррентных слоёв: выход $h_t^{(l)}$ одного слоя идёт на вход следующего.
- Повышает выразительность, но усложняет обучение.

RNN pooling

Чтобы получить фиксированный вектор из последовательности:

- брать последнее скрытое состояние h_T,
- делать среднее/макс по времени: $\text{pool}(h_{1:T})$,
- attention pooling (взвешенное суммирование).

Bidirectional RNN (Bi-RNN)

- Два прохода: слева направо и справа налево.
- Для каждого t объединяем $\overrightarrow{h_t}$ и $\overleftarrow{h_t}$.
- Хорошо для задач разметки (NER), когда доступен весь контекст.
- Неприменимо для строгого онлайн-предсказания «в будущее» без задержки.

45 Билет 45. Операция свёртки

Свёртка как операция

В 1D (непрерывно): $(f * g)(t) = \int f(\tau)g(t-\tau)\,d\tau$. В дискретном 2D (как в CNN) обычно используют корреляцию:

$$(Y)_{i,j} = \sum_{u,v} K_{u,v}\, X_{i+u,j+v}.$$

В ML часто называют это "convolution", хотя строгая свёртка имеет переворот ядра.

Local receptive fields

Каждый выходной пиксель зависит только от локального окна входа $\Rightarrow$ параметров меньше, чем в dense, и есть индуктивное смещение на локальные паттерны.

Выбор размера ядра

- Большие ядра видят больший контекст, но дороже по вычислениям.
- Часто используют несколько слоёв 3×3 вместо одного большого.

1x1 свёртки

- Меняют число каналов (mixing по каналам) без смешивания по пространству.
- Альтернатива FC по каналам в каждой позиции.

Гиперпараметры и число параметров

Пусть C_{in} входных каналов, C_{out} выходных, ядро $k_h \times k_w$:

$$\#\text{params} = C_{out} \cdot C_{in} \cdot k_h k_w + C_{out} \text{ (bias)}.$$

Размер выхода

Для одной оси (аналогично для H и W):

$$\text{out} = \left\lfloor \frac{\text{in} + 2P - K}{S} \right\rfloor + 1,$$

где P — padding, S — stride, K — kernel size.

Разделимые и depthwise (*)

- **Spatial separable**: разложение $k \times k$ на $k \times 1$ и $1 \times k$.
- **Depthwise convolution**: свёртка отдельно по каждому каналу + pointwise 1×1 для смешивания.
- Это снижает вычисления и параметры (MobileNet-подобные архитектуры).

Память и сравнение с Linear

Dense слой растёт как $O(d \cdot m)$ по параметрам, свёртка — как $O(C_{in}C_{out}k^2)$ и не зависит от размера изображения напрямую.

Матричная форма (*)

Свёртку можно переписать как умножение большой разреженной матрицы на вектор (im2col), что удобно для реализации через GEMM.

46 Билет 46. Свёрточный слой

Структура фильтров в обученной сети

- На ранних слоях часто появляются фильтры типа границ (edge detectors), цветовых контрастов.
- На более глубоких слоях — композиции: текстуры, части объектов, более абстрактные паттерны.

Это интуитивная картина и зависит от данных/архитектуры.

CNN artifacts (*)

Артефакты могут возникать из-за stride/padding, транспонированных свёрток (checkerboard artifacts), неудачных апсемплингов, неинвариантных предобработок.

Что означают градиенты

- Градиент по весу фильтра показывает, как изменение конкретного коэффициента ядра влияет на loss.
- Градиент по входу показывает «куда» loss чувствителен во входном изображении (saliency maps).
- Важно помнить: градиенты — локальная информация и не всегда дают «человеческое объяснение».

47 Билет 47. Pooling layer

Max и average pooling

- **Max pooling**: берём максимум по окну $k \times k$.
- **Average pooling**: берём среднее по окну.

Зачем нужен pooling

- Уменьшение пространственного размера $\Rightarrow$ меньше вычислений.
- Некоторая инвариантность к небольшим сдвигам/деформациям.
- Увеличение receptive field при фиксированном числе параметров.

Adaptive pooling

Adaptive pooling выбирает размер окна так, чтобы получить заданный размер выхода (например, 1×1). Полезно, чтобы сеть могла принимать вход разного размера и давать фиксированный вектор признаков перед классификатором.

Нюансы

Сегодня иногда заменяют pooling на stride-свёртки или другие приёмы, но базовая идея остаётся важной.

48 Билет 48. Датасет ImageNet: параметры

Что обычно имеют в виду

Чаще всего под «ImageNet» подразумевают набор ILSVRC-2012 для классификации:

- около 1.2 млн изображений в train,
- 50 тыс в validation,
- 1000 классов.

Размер изображений

Оригинальные изображения разного размера; в обучении обычно делают препроцессинг:

- ресайз по короткой стороне + кроп, или
- фиксированный размер входа модели (часто 224×224 для многих архитектур).

Зачем это знать

ImageNet — исторический бенчмарк, на котором сравнивали архитектуры (AlexNet→VGG→ResNet→E
и демонстрировали силу предобучения (transfer learning).

49 Билет 49. Архитектуры классификации изображений

AlexNet

- Одна из первых «глубоких» CNN, взорвавшая ImageNet (2012).
- Идеи: ReLU, dropout, data augmentation, использование GPU.
- Ограничения: сейчас устарела, много параметров и не лучшая эффективность.

VGG

- Простая идея: много слоёв 3×3 + pooling.
- Плюс: простота, хороша как «feature extractor».
- Минус: огромная параметризация, дорогая по памяти/compute.

Inception (GoogLeNet)

- Inception-блок: параллельные ветки (1x1, 3x3, 5x5) + конкатенация.
- 1x1 используется для «бутылочного горлышка» (уменьшение каналов).
- Плюсы: эффективнее по параметрам, сильные результаты.
- Минусы: архитектурно сложнее.

ResNet

- Ключ: residual connections $y = x + F(x)$.

- Решает проблему обучения очень глубоких сетей (затухание градиентов/деградация качества).
- Стала стандартным бэкбоном для CV.

SENet

- Squeeze-and-Excitation: внимание по каналам (reweight каналов).
- Улучшает качество при небольшом росте вычислений.

EfficientNet

- Идея compound scaling: согласованно масштабировать глубину/ширину/разрешение.
- Хороший компромисс точность–эффективность.
- Минус: чувствительность к реализации и режимам обучения.

Общее, что можно сказать на экзамене

- Эволюция: рост глубины + стабилизация обучения + эффективность.
- Проблемы: вычислительная стоимость, требование данных, устойчивость к сдвигам, интерпретация.

50 Билет 50. Механизм внимания (Attention)

Seq2seq на RNN без внимания

Encoder читает вход и сжимает всю информацию в один вектор (последнее состояние), decoder генерирует выход. Проблема: «бутылочное горлышко» для длинных последовательностей.

Зачем нужно внимание

Attention позволяет decoder на каждом шаге «смотреть» на разные части входа, а не только на один вектор.

Cross-attention (RNN attention)

На шаге t decoder имеет query q_t; encoder даёт keys/values k_i, v_i для позиций i. Считаем веса:

$$\alpha_{t,i} \propto \mathrm{score}(q_t, k_i), \qquad c_t = \sum_i \alpha_{t,i} v_i,$$

где c_t — контекст для генерации следующего токена.

Self-attention как «частный случай»

Self-attention: query, key, value берутся из одной последовательности. Это позволяет каждой позиции взаимодействовать с другими позициями напрямую.

Другие виды внимания

- Channel/spatial attention в CV (например, SENet).
- Additive (Bahdanau) vs dot-product (Luong) attention.
- Multi-head attention (несколько «проекций» внимания).

51 Билет 51. Архитектура Transformer

Почему трансформер

Self-attention позволяет моделировать дальние зависимости без рекурсии и хорошо параллелится на GPU.

Positional encoding

Поскольку self-attention сам по себе не знает порядок, добавляют позиционную информацию:

- фиксированные синусоиды (classic Transformer),
- обучаемые позиционные эмбеддинги,
- относительные позиционные схемы (в современных вариантах).

Encoder и Decoder

- **Encoder-блок**: self-attention + FFN (MLP) + residual + normalization.
- **Decoder-блок**: masked self-attention + cross-attention к encoder + FFN + residual + norm.

Residual connections и normalization

- Residual: $x \leftarrow x + \text{sublayer}(x)$ улучшает градиенты.
- Normalization (LayerNorm) стабилизирует обучение.

Masked inference

В языковом моделировании запрещаем «смотреть в будущее»: в attention матрице применяем маску, зануляющую связи с будущими позициями.

Ключевые плюсы/минусы

- Плюсы: параллельность, сильные результаты.
- Минусы: attention $O(T^2)$ по длине последовательности; для очень длинных T нужны оптимизации.

52 Билет 52. Geometric Machine Learning

Проклятие размерности

В высоких размерностях:

- расстояния «схлопываются» (контрасты между близкими и дальними уменьшаются),

- плотность данных становится крайне разреженной,
- нужны экспоненциально большие данные для покрытия пространства.

Manifold assumption

Хотя данные лежат в $\mathbb{R}^d$, реально они часто концентрируются около низкоразмерного многообразия (manifold) внутри $\mathbb{R}^d$.

Latent space

Латентное пространство — скрытое представление меньшей размерности, в котором структура данных проще (кластеры, линейные зависимости). Эмбеддинги и автоэнкодеры строят такие пространства.

Intrinsic dimension

Внутренняя размерность — эффективная размерность многообразия, на котором лежат данные. Это объясняет, почему снижение размерности и локальные методы могут работать.

53 Билет 53. Снижение размерности: IsoMap, t-SNE, UMAP

IsoMap

- Строим граф ближайших соседей.
- Оцениваем геодезические расстояния как кратчайшие пути по графу.
- Делаем MDS (multidimensional scaling) по этим расстояниям.
- Хорошо для «гладких» многообразий, но чувствительно к выбору графа.

t-SNE

- Сохраняет локальные соседства: превращает расстояния в вероятности соседства в high-D и low-D.
- Минимизирует KL-дивергенцию между распределениями соседства.
- Отлично визуализирует кластеры, но глобальные расстояния/масштабы не интерпретировать буквально.
- Параметр perplexity влияет на «масштаб» локальности.

UMAP

- Тоже сохраняет локальную структуру, основан на идеях топологии и графов.
- Часто быстрее t-SNE и лучше сохраняет некоторую глобальную структуру.
- Имеет параметры `n_neighbors`, `min_dist`.

Практическая рекомендация

Для визуализации: t-SNE/UMAP; для дальнейшего ML-пайплайна чаще используют PCA как стабильный линейный метод, либо обучаемые эмбеддинги.

54 Билет 54. Кластеризация

Постановка

Кластеризация — разбиение объектов на группы без учителя так, чтобы внутри группы объекты были похожи.

K-means

- Ищем K центроидов, минимизируя сумму квадратов расстояний до ближайшего центроида.
- Алгоритм Ллойда: (1) назначение кластера, (2) пересчёт центроидов, до сходимости.
- Минусы: нужно задавать K, чувствителен к инициализации и масштабам, предполагает «сферические» кластеры.

DBSCAN

- Основан на плотности: кластеры — области высокой плотности.
- Параметры: ϵ (радиус), `min_samples`.
- Плюсы: находит кластеры произвольной формы, выделяет шум.
- Минусы: сложно подобрать ϵ в высоких размерностях.

Иерархическая кластеризация

- Агломеративная: начинаем с одиночных кластеров и постепенно объединяем.
- Нужно выбрать linkage: single/complete/average/Ward.
- Результат — дендрограмма, можно выбрать число кластеров «срезом».

Метрики кластеризации

- Без меток: silhouette score, Davies–Bouldin, Calinski–Harabasz.
- С метками (для оценки): ARI (Adjusted Rand Index), NMI (Normalized Mutual Information).

55 Билет 55. Ранжирование

Постановка

В ранжировании для запроса q и набора документов/товаров $\{d_i\}$ нужно упорядочить их по релевантности. Отличие:

- от регрессии: важен порядок, а не точное значение,
- от классификации: есть много кандидатов и позиционный эффект.

Offline-метрики

- **MRR**: 1/rank первого релевантного.
- **mAP**: средняя precision по позициям релевантных, затем среднее по запросам.
- **NDCG**: учитывает ранги и «градуированную» релевантность:

$$\text{DCG@k} = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)}, \quad \text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}.$$

Online-метрики

- Измеряются по логам: CTR, конверсия, время до клика, retention.
- Корректное сравнение требует A/B тестов и контроля смещений (позиционные/selection bias).

Metric tree и North Star

В продукте верхнеуровневая метрика (north star) раскладывается на подметрики, которые ближе к моделям (например, улучшение NDCG → рост CTR → рост выручки).

RankNet loss

RankNet обучает модель сравнивать пары документов:

- Предсказывает score $s(d)$.
- Для пары (i, j) хотим $s_i > s_j$, если d_i релевантнее.
- Потеря: логистическая по разности $s_i - s_j$.

Что предсказывает: относительное предпочтение/вероятность, что i лучше j.

Lambda gradients (*)

Идея LambdaRank/LambdaMART: менять градиенты так, чтобы напрямую оптимизировать NDCG-подобные метрики.

Другие функции потерь (просто назвать)

ListNet, ListMLE, pairwise hinge, softmax cross-entropy по списку и др.

56 Билет 56. Обработка текстов (*)

Лемматизация и стемминг

- **Стемминг**: грубо отрезаем окончания (быстро, но иногда ломает смысл).
- **Лемматизация**: приводим слово к нормальной форме (точнее, но дороже).

Bag of Words

- Представляем документ как вектор частот слов (или TF-IDF).
- Плюс: простота, работает с линейными моделями.
- Минус: теряется порядок слов, высокая размерность и разреженность.

n-grams

- Учитывают последовательности из n токенов (би-граммы, три-граммы).
- Улучшают качество по сравнению с unigram BoW, но размер словаря растёт.

Современная практика

Вместо BoW часто используют эмбеддинги и трансформеры, но BoW — отличный базовый бейзлайн и часто неплохо работает на небольших данных.

57 Билет 57. Машинный перевод (*)

Структура датасета

Параллельный корпус: пары предложений (source, target) на двух языках. Иногда есть дополнительные метаданные (домен, стиль).

Seq2seq NMT на RNN

- Encoder кодирует source-последовательность в скрытые состояния.
- Decoder генерирует target токены авто-регрессивно.
- Основная проблема базовой схемы без attention: bottleneck (всё в один вектор).
- Attention решает проблему (см. билет 50).

Beam search

При генерации нужно выбрать последовательность токенов, максимизирующую вероятность. Жадный выбор (greedy) часто хуже. Beam search хранит B лучших гипотез на каждом шаге (beam width B).

BLEU

Метрика качества перевода, основанная на совпадении n-грамм с эталонным переводом + штраф за слишком короткий перевод (brevity penalty). Нюанс: BLEU коррелирует с качеством не идеально, но исторически популярна.

58 Билет 58. Экспоненциальное сглаживание (*)

Формула (простое сглаживание)

Для временного ряда y_t:

$$s_t = \alpha y_t + (1 - \alpha) s_{t-1}, \qquad \alpha \in (0, 1).$$

s_t — сглаженное значение.

Почему “экспоненциальное”

Разворачивая рекурсию, получаем:

$$s_t = \alpha y_t + \alpha(1 - \alpha) y_{t-1} + \alpha(1 - \alpha)^2 y_{t-2} + \dots$$

То есть веса прошлых наблюдений убывают экспоненциально.

Где применяют

Быстрый базовый метод прогнозирования и сглаживания, иногда как компонент более сложных моделей (Holt-Winters и т.п.).

59 Билет 59. Методы оптимизации (*)

GD и SGD

- GD: полный градиент по всему датасету.
- SGD: градиент по мини-батчу, быстрее и масштабируемее.

Momentum

Добавляем скорость v:

$$v_t = \beta v_{t-1} + (1-\beta)\nabla L(w_t), \qquad w_{t+1} = w_t - \eta v_t.$$

Интуиция: сглаживаем шум и ускоряем движение по устойчивому направлению.

RMSProp

Поддерживаем экспоненциальное среднее квадратов градиентов:

$$s_t = \rho s_{t-1} + (1-\rho) g_t^2, \qquad w_{t+1} = w_t - \eta \frac{g_t}{\sqrt{s_t} + \epsilon}.$$

Adam

Комбинирует momentum (первый момент) и RMSProp (второй момент), часто является дефолтом в DL.

Память (грубо)

- SGD: хранит только параметры w (и градиент).
- Momentum: + один буфер v (ещё один вектор размера параметров).
- RMSProp: + один буфер s.
- Adam: + два буфера (первый и второй моменты) $\Rightarrow$ примерно в 3 раза больше памяти под оптимизатор, чем SGD.

60 Билет 60. Inductive bias в Deep Learning (*)

Что такое inductive bias

Индуктивное смещение — предположения модели/обучения о том, какие решения «правильные». Они помогают обобщать за пределы train.

Регуляризация

- Weight decay (L2) — предпочитает малые веса.
- Early stopping — ограничивает «выучивание шума».
- Data augmentation — задаёт инвариантности (особенно в CV).

Dropout

- Во время обучения случайно зануляем активации с вероятностью p.
- Это похоже на ансамбль многих подсетей и снижает переобучение.

Batch Normalization и Layer Normalization

- BatchNorm нормализует по батчу (по статистикам батча), ускоряет и стабилизирует обучение.
- LayerNorm нормализует по признакам внутри одного объекта; удобна для трансформеров и маленьких батчей.

Разница train vs eval

- Dropout: **train** — включён; **eval** — выключен (используем все нейроны).
- BatchNorm: **train** — использует статистики батча и обновляет running mean/var; **eval** — использует накопленные running mean/var (фиксированные).

Теоретический минимум и вопросы с тестов

61 Теоретический минимум: краткие ответы

Ниже — ответы в формате «что нужно уметь быстро сказать», чтобы закрыть минимальные требования программы.

1. **Регрессия и классификация.** Регрессия: $y \in \mathbb{R}$, минимизируем ошибку численного предсказания (MSE/MAE и т.д.). Классификация: y дискретен, предсказываем класс или вероятность; метрики accuracy/F1/AUC и т.д.
2. **Unsupervised learning.** Нет меток y, ищем структуру: (1) кластеризация (k-means/DBSCAN), (2) снижение размерности (PCA/t-SNE/UMAP).
3. **i.i.d. данные.** Независимые и одинаково распределённые: $p(x_{1:n}) = \prod_i p(x_i)$. Используем в MLE (произведение правдоподобий), в теории обобщения и в построении доверительных оценок.
4. **Naive Bayes.** $p(y \mid x) \propto p(y) \prod_j p(x_j \mid y)$. Наивность: условная независимость признаков при фиксированном y.
5. **Линейная регрессия и градиент MSE.** $\hat{y} = Xw$, $\mathcal{L}(w) = \frac{1}{n}\|Xw - y\|^2$, $\nabla_w \mathcal{L} = \frac{2}{n} X^\top (Xw - y)$.
6. **Шаг градиентного спуска и большие датасеты.** $w \leftarrow w - \eta \nabla L(w)$. Для больших данных: mini-batch SGD, потоковое обучение, distributed training.
7. **Likelihood и где используется MLE.** Likelihood — $p(\text{data} \mid \theta)$. MLE — оценивание параметров вероятностной модели (логрег, гауссовы модели, HMM и др.).
8. **Cross-validation и число фолдов.** K-fold CV: среднее качество по K разбиениям. Больше $K \Rightarrow$ меньше смещение оценки, но больше вычисления и выше дисперсия (часто 5 или 10).
9. **Over/underfitting.** Underfit: плохо везде. Overfit: хорошо на train, плохо на val. Смотрим кривые обучения, CV, gap train-val.
10. **Параметры vs гиперпараметры.** Параметры обучаются (веса), гиперпараметры задаются (глубина дерева, λ). Примеры: линейная модель w vs λ; дерево: пороги vs max_depth.
11. **Регуляризация и L1/L2.** Добавка к loss: $+\lambda\|w\|_p$. L2 стягивает веса, L1 даёт разреженность. Не единственный способ ограничить решение: early stopping, ограничение

модели, data augmentation.

12. **Регуляризует ли L2 bias?** Обычно нет: intercept отражает базовый уровень и его регуляризация часто ухудшает; кроме того, он зависит от масштабирования.
13. **Зачем нормализовать перед линейной моделью?** Чтобы признаки были сопоставимы, улучшить обусловленность $X^\top X$ и ускорить GD; особенно критично для kNN, SVM, регуляризации.
14. **Линейная классификация и маржа.** $\hat{y} = \text{sign}(w^\top x + b)$, маржа $\gamma = y(w^\top x + b)/\|w\|$.
15. **Precision и recall.** $P = \frac{TP}{TP+FP}$, $R = \frac{TP}{TP+FN}$; выбираем порог, балансируем в зависимости от стоимости FP/FN.
16. **Дисбаланс 95/5.** Accuracy не подходит; используем PR-AUC, balanced accuracy, F1; взвешивание классов, ресэмплинг, подбор порога.
17. **ROC AUC и ROC.** ROC: TPR vs FPR по порогам. AUC: площадь, вероятность ранжировать positive выше negative.
18. **Логистическая потеря и связь с MLE.** Logloss — NLL Bernoulli; MLE для логрегации приводит к минимизации logloss.
19. **SVM: идея и функционал (separable).** Максимизация маржи: $\min \frac{1}{2}\|w\|^2$ при $y_i(w^\top x_i + b) \geq 1$.
20. **Жадный алгоритм дерева.** На каждом узле выбираем лучший сплит по критерию (entropy/gini/MSE), рекурсивно строим поддеревья.
21. **Почему дерево может сделать нулевую train-ошибку при уникальных объектах?** Можно разделять пока в листе останется 1 объект: дерево «запоминает».
22. **Метка в листе (классификация).** Обычно majority vote; вероятности по частотам.
23. **Ответ в листе (регрессия).** Обычно среднее (MSE-оптимально) или медиана (MAE-робастно); зависит от выбранного критерия.
24. **Bagging.** Ансамбль моделей на бутстрэпе с усреднением/голосованием для снижения variance.
25. **Random Forest vs Bagging деревьев.** RF добавляет случайный поднабор признаков в узлах, сильнее декоррелирует деревья.
26. **Как обучаются базовые алгоритмы в бустинге.** Последовательно: каждый новый базовый алгоритм учится исправлять ошибки предыдущего (антиградиент).
27. **Backprop и vector-by-vector derivative.** Распространение градиентов по графу через цепное правило; на практике считаются произведения вида $g^\top J$.
28. **Свёртка и conv layer.** Локальные ядра, общие веса; формула дискретной корреляции; параметры $C_{out} \cdot C_{in} \cdot k^2$.
29. **Почему dense хуже для картинок, а CNN лучше?** Dense игнорирует локальность и даёт огромное число параметров; CNN использует локальные receptive fields и weight sharing, инвариантность к сдвигам.
30. **Vanilla RNN.** $h_t = \phi(W_x x_t + W_h h_{t-1})$, обучаем BPTT; проблемы градиентов.
31. **Dropout.** Случайное зануление активаций при обучении, уменьшает переобучение.
32. **Dropout и BatchNorm на inference.** Dropout выключен. BatchNorm использует накопленные running статистики, не статистики батча.
33. **Постановка PCA.** Найти ортонормальные направления максимальной дисперсии/лучшее приближение ранга k (через SVD).

62 Вопросы с тестов: что отвечать в одну-две минуты

1. 3–5 задач ML: классификация, регрессия, кластеризация, ранжирование, детекция аномалий.
2. MLE: $\hat{\theta} = \arg\max \prod_i p(x_i|\theta)$, iid $\Rightarrow$ сумма логов, NLL.
3. Регрессия: предсказать $y \in \mathbb{R}$; линейная регрессия добавляет $\hat{y} = w^\top x + b$ и аналитиче-

ское решение.

4. Наивность NB: условная независимость признаков.
5. OLS: $(X^\top X)^{-1}X^\top y$; проблемы: d большой, $X^\top X$ плохо обусловлена/необратима.
6. Гаусс–Марков: OLS — BLUE при стандартных допущениях.
7. Регуляризация: L1/L2/elastic net; ridge решение $(X^\top X + n\lambda I)^{-1}X^\top y$.
8. 3–5 loss для регрессии: MSE, MAE, Huber, quantile loss, log-cosh.
9. Переобучение: train ↑, val ↓; обнаружение по gap и CV.
10. Параметры vs гиперпараметры: обучаются vs выбираются; примеры.
11. 3–5 способов валидации: holdout, K-fold CV, stratified, group split, time split.
12. kNN и масштабы: доминирование расстояния; решение — scaling.
13. Метрика падает при добавлении фичи: шум/утечка/корреляции/переобучение, плохая валидация.
14. Что предсказывает логрег: вероятность класса (логиты/score).
15. Сигмоида: $\sigma(z) = \frac{1}{1+e^{-z}}$.
16. OvR/OvO: K моделей vs $K(K-1)/2$.
17. Для ROC-AUC нужны скоры/вероятности, а не только метки.
18. Precision/Recall: формулы и смысл.
19. Размер бутстрэпа: n объектов с возвращением.
20. RF vs Bagging: случайные признаки в узлах + бутстрэп.
21. Таргет следующего дерева в бустинге: антиградиент (остатки для MSE).
22. Библиотеки бустинга: XGBoost, LightGBM, CatBoost.
23. Логистическая потеря: $-\big(y\log p + (1-y)\log(1-p)\big)$.
24. 3–5 активаций: ReLU, tanh, sigmoid, GELU, LeakyReLU.
25. Критерии сравнения активаций: насыщение, гладкость, диапазон, вычисления, «dead ReLU».
26. BatchNorm на предсказании: использует running mean/var, не статистики батча.
27. Momentum: $v_t = \beta v_{t-1} + (1-\beta)g_t$ (или $v_t = \beta v_{t-1} + g_t$ в другой записи).
28. PCA новые признаки: проекции на главные компоненты.
29. PCA как оптимизация: максимизация дисперсии по осям / минимизация ошибки реконструкции.
30. SVM геометрия: максимальная маржа, опорные векторы, hinge loss.
31. Loss SVM: hinge.
32. Энтропия: $H(p) = -\sum p\log p$.
33. Функция в узле дерева: критерий impurity (gini/entropy) или MSE, выбираем лучший сплит.
34. Актив. в RNN: tanh/sigmoid, потому что ограничивают значения и исторически стабильны; плюс gating в LSTM/GRU.
35. Размер выхода conv над 8×5 с ядром 3×3 (stride=1, padding=0): $(8-3+1)\times(5-3+1) = 6\times3$ на каждый фильтр; с 4 ядрами $\Rightarrow$ 4 канала.
36. Паттерны по глубине CNN: от простых границ к сложным частям и объектам.
37. Padding и stride: padding расширяет вход, stride задаёт шаг; влияют на размер выхода и receptive field.
38. Pooling: уменьшает размер и даёт инвариантность; иногда заменяют stride-conv.
39. Проблема глубоких сетей: деградация/затухание градиентов; решает ResNet (residual).
40. Минусы стат. МП: требует много ручных правил/фич, хуже переносимость; NMT обучается end-to-end.
41. Схема seq2seq без внимания: encoder → один вектор → decoder.
42. Beam search: ищет более вероятную последовательность, чем greedy.
43. Основная метрика перевода: BLEU = совпадение n-грамм + brevity penalty.
44. Attention score в seq2seq: скалярная функция совместимости query и key (dot/additive),

нормируем softmax.